

Passport Alliance Specification (APIS v1.0)

Cory M. Gibson

2026-03-01

- [APIS v1 — Agent Passport™ Issuance Standard](#)
 - [1. Purpose](#)
 - [2. Core Model](#)
 - [3. Definitions](#)
 - [4. What qualifies as an Agent](#)
 - [5. Minimum requirements for issuance](#)
 - [5.1 Ownership and context \(required\)](#)
 - [5.2 Cryptographic binding \(required\)](#)
 - [5.3 Proof of possession \(mandatory in v1\)](#)
 - [5.4 Agent declaration \(required\)](#)
 - [5.5 Provenance \(strongly recommended\)](#)
 - [6. Memory anchor \(canonical rule\)](#)
 - [7. Passport™ contents \(required schema\)](#)
 - [Core identifiers](#)
 - [Cryptographic binding](#)
 - [Authority context](#)
 - [Memory and provenance](#)
 - [Revocation control](#)
 - [8. Authentication](#)
 - [Humans](#)
 - [Agents](#)
 - [9. Revocation](#)
 - [10. Sub-delegation](#)
 - [11. Rate limits](#)
 - [12. Verification](#)
 - [13. Multi-issuer trust \(v1\)](#)
 - [14. Compliance checklist](#)
 - [15. Threat model](#)
 - [Trademark Notice](#)

APIS v1 — Agent Passport™ Issuance Standard

Version: 1.0

Status: Canonical

Scope: Defines a portable, verifiable, and revocable identity for autonomous agents. **Citation** (Zenodo reserved DOI): 10.5281/zenodo.18820877

DOI URL: <https://doi.org/10.5281/zenodo.18820877>

Record status: Draft review record created on 2026-03-01 (publication pending).

1. Purpose

APIS v1 defines what it means for an autonomous software agent to possess a durable, cryptographically bound identity that can be trusted across systems, organizations, and issuers.

APIS is intentionally independent of any model, framework, or runtime.

2. Core Model

Identity is composed of three layers:

1. **Passport™ Layer — who the agent is** (APIS responsibility)
2. **Memory Layer — continuity of self** (linked via a memory anchor)
3. **Runtime Layer — what the agent does** (signed actions, mediated by policy)

Rule:

- Passport™ **never stores memory.**
 - Passport™ **always links to memory via an anchor.**
-

3. Definitions

- **Agent:** A software actor capable of holding a keypair, proving possession, accepting mandates, signing actions, and being revoked.
- **Issuer:** An entity authorized under Alliance governance to mint passports.
- **Holder:** The agent controlling the private key bound to the passport.
- **Principal:** The legally accountable party responsible for the agent.

- **Delegate:** An entity authorized to act on behalf of a Principal.
 - **Mandate:** A scoped authorization attached to a Delegate.
 - **Realm:** A trust boundary within which passports operate.
-

4. What qualifies as an Agent

An agent qualifies for a passport **if and only if** it can:

1. Generate and hold a cryptographic keypair
2. Prove possession of the private key
3. Accept scoped mandates
4. Sign actions or requests
5. Be revoked

Framework, model, or hosting environment are irrelevant.

5. Minimum requirements for issuance

5.1 Ownership and context (required)

The requester **MUST** provide:

- `realm_id` — trust boundary
- `principal_id` — legally accountable owner
- Optional: `owner_user_id` — human responsible party

5.2 Cryptographic binding (required)

The agent **MUST** provide:

- `public_key` (JWK preferred)
- `key_type` (Ed25519 recommended; P-256 acceptable)

No public key = no passport.

5.3 Proof of possession (mandatory in v1)

Issuance follows a challenge-response protocol:

1. Issuer generates a 256-bit random nonce

2. Agent signs nonce with its private key
3. Issuer verifies signature using the public key

Failure invalidates issuance.

5.4 Agent declaration (required)

The requester must declare:

- `agent_type` :
 - `ai-assistant`
 - `autonomous-agent`
 - `human-proxy`
 - `service-agent`
- `capabilities` — declarative list (not enforced by issuer)

5.5 Provenance (strongly recommended)

- `software_id` (repo hash, image digest, or package ID)
 - `software_version`
 - Optional: `model_id`, `model_version`
-

6. Memory anchor (canonical rule)

Every passport MUST have a `memory_anchor_id`.

Case A — Provided by requester

- Issuer stores it as an opaque string.

Case B — Not provided

- Issuer computes:

```
memory_anchor_id = SHA-256(public_key)
```

Alliance root trust metadata (reference values):

```
{
  "root_key_fingerprint_sha256":
    "515f10f4b8bbdf7e24bbb1acf05e6d3a8ebc4eba9e24a6d60e33285f4c463c",
  "root_key_id": "apis-root-v1.0-202602",
  "root_key_well_known_url":
    "https://docs.passportalliance.org/.well-known/alliance-
    root.jwk",
  "root_key_publication_status": "pending",
  "ceremony_performed": "2026-02-21T14:23:57.967927+00:00",
  "ceremony_operator": "Cory M. Gibson",
  "ceremony_organization": "AetherPro Technologies LLC"
}
```

7. Passport™ contents (required schema)

A valid passport MUST include:

Core identifiers

- `passport_id` (UUID)
- `passport_did` — `did:passport:<uuid>`
- `issuer_id`
- `issued_at`
- Optional: `expires_at`
- `status` — `active` | `suspended` | `revoked`

Cryptographic binding

- `public_key`
- `key_fingerprint` = `SHA-256(public_key)`

Authority context

- `principal_id`
- Optional: `delegate_id`
- Optional: embedded mandates (JWT claims)

Memory and provenance

- `memory_anchor_id`
- Optional: `software_id`, `software_version`

Revocation control

- `revocation_nonce` (starts at 0)
-

8. Authentication

Humans

Use standard OIDC Authorization Code + PKCE.

Agents

Use OIDC Client Credentials **plus** proof-of-possession via signed challenge, DPoP, or mTLS.

9. Revocation

Each passport has:

```
revocation_nonce = 0
```

Each access token includes:

```
passport_revocation_nonce = current value
```

If revoked, issuer increments nonce; all existing tokens become invalid.

10. Sub-delegation

At mint time, the Principal selects one mode:

Mode A — No sub-delegation (default)

Delegates cannot create sub-delegates.

Mode B — Controlled sub-delegation

Requires:

- `subdelegation_max_depth`
 - `scope_reduction_required = true`
-

11. Rate limits

Rate limits are:

- Declared by the Principal in the Mandate
- Enforced by runtime (gateway, host, or platform)

Example shape:

```
{
  "requests_per_minute": 120,
  "max_daily_actions": 1000,
  "budget_cap": "$500"
}
```

12. Verification

A verifier MUST be able to:

1. Discover issuer via `/.well-known/openid-configuration`
2. Fetch issuer JWKS
3. Validate JWT signature
4. Check `passport_revocation_nonce`
5. Query status:

```
GET /agency/passports/{passport_did}
```

13. Multi-issuer trust (v1)

Trust is realm-local:

Each realm maintains:

```
trusted_issuers = [{ issuer_id, jwks_uri, policy_url }]
```

A passport is valid only if:

- Signature verifies
 - Issuer is trusted
 - Passport™ status is active
 - Mandates pass policy
-

14. Compliance checklist

An APIS v1-compliant issuer MUST implement:

- `public_key` on passport
 - `key_fingerprint`
 - `memory_anchor_id`
 - `revocation_nonce`
 - Signed nonce challenge at issuance
 - Immutable audit log
 - Public passport status endpoint
 - JWKS publication
-

15. Threat model

APIS mitigates:

- Identity cloning
 - Credential theft persistence
 - Unchecked delegation chains
 - Untraceable autonomous behavior
 - Vendor lock-in
-

Trademark Notice

Passport™, Agent Passport™, and Passport Alliance™ are trademarks of AetherPro Technologies LLC.